

Trabajo Práctico Final — Entregable 3

Caso: Plataforma Magic (*MagicDB*)

Autor: Barrios, Julieta Luján

Materia: Base de Datos - 3er año Ingeniería Informática

Año: 2026

Contenido de esta entrega

Este entregable corresponde a la **capa de acceso** (función escalar, vistas y stored procedures) + **dashboard** del TP Final.

Archivo	Contenido
SQLQuery4 (vistas).sql	Función escalar <code>fn_SemanaMes</code> + 8 vistas analíticas, una por pregunta de negocio.
SQLQuery5 (índices).sql	3 índices no agrupados justificados, cada uno atado a una consulta concreta de la capa de acceso.
SQLQuery6 (SP).sql	3 stored procedures con parámetros funcionales y validación de entrada.
Trabajo Práctico Final - Grupo 3 - Entregable 3.pdf	Este documento: visión general, cumplimiento de consigna y justificación de cada decisión técnica.
magicdb-dashboard.json	Archivo con el dashboard de Grafana (archivo JSON exportable).

1. Cómo ejecutar

```
-- Pre-requisito: MagicDB creada y con datos cargados (Entregable 2).

-- 1. Ejecutar en SSMS sobre MagicDB, en este orden exacto:
--   SQLQuery4 (vistas).sql ← primero: crea fn_SemanaMes antes
--   que las vistas que la usan
--   SQLQuery5 (indices).sql ← segundo: los índices referencian
--   tablas ya existentes
--   SQLQuery6 (SP).sql ← tercero: los SPs referencian vistas ya
--   creadas

-- 2. Levantar el dashboard con Grafana (Docker):
--   a. Ejecutar en terminal:
--       docker run -d -p 3000:3000 --name grafana grafana/grafana
--   b. Abrir http://localhost:3000 (usuario: admin / contraseña:
--   admin)
--   c. Ir a Connections → Data Sources → Add data source →
--   Microsoft SQL Server
```

```
-- d. Completar: Host = localhost\SQLEXPRESS (o la instancia
correspondiente)
--           Database = MagicDB
--           Authentication = SQL Server Authentication
-- e. Click en Save & Test – debe aparecer "Database Connection
OK"
-- f. Ir a Dashboards → Import → subir el archivo
dashboard/magicdb-dashboard.json
-- g. Cuando pida data source, seleccionar el creado en el paso
(d)
```

Nota sobre el orden de ejecución de SQLQuery4: la función escalar fn_SemanaMes se crea al inicio del archivo, antes de las vistas v_VentasPorSemanaMes y v_SuscripcionesPorSemanaMes que la invocan. Si se ejecutara solo la sección de vistas sin la función, esas dos vistas fallarían. El archivo está diseñado para ejecutarse completo de arriba a abajo.

2. Cumplimientos de a consigna

2.1 Mínimos Cuantitativos

Requisitos de la consigna	Mínimo	Entregado
Vistas con propósito claro	5	14 — al menos una por cada pregunta de negocio planteada en E1
Stored procedures con parámetros	3	3 con validación de entrada en todos
Uso de CTE	1	4 usos genuinos: v_RequestAlertasPorSuscripcion, v_ConversionCompradoresAssets, v_PicosTraficoAlertas, v_UpgradesDowngradesSuscripciones
Uso de función de ventana	1	6 usos distintos: SUM() OVER en v_RequestAlertasPorSuscripcion; LAG() en v_UpgradesDowngradesSuscripciones; RANK() en v_TendenciaVentasMensual, v_VentasPorSemanaMes, v_SuscripcionesPorSemanaMes y sp_BuscarComprasAssets
Índices	Opcionales	3 índices cubrientes justificados
Funciones de usuario	Opcionales	1 función escalar (fn_SemanaMes)
Indicadores en dashboard	5	8 paneles — uno por pregunta de negocio

2.2 Restricción dura — el dashboard NO consulta tablas directamente

Verificación: ningún panel de Grafana ejecuta FROM <Tabla> directamente. Todas las queries del dashboard apuntan a vistas (FROM v_*) o ejecutan stored procedures (EXEC sp_*).

Listado completo de queries del dashboard:

Panel Grafana	Query
Fila 1 – Pregunta de Negocio 1: Popularidad de assets	<pre> SELECT TipoAsset, SUM(CantidadAdquisiciones) AS Ventas, SUM(TotalIngresosGenerados) AS IngresosTotales FROM v_PopularidadAssetsTienda GROUP BY TipoAsset ORDER BY Ventas DESC </pre>
Fila 2 – Pregunta de Negocio 2: Alertas por plan	<pre> SELECT NombrePlan, CantidadAlertasProcesadas FROM v_RequestAlertasPorSuscripcion ORDER BY CantidadAlertasProcesadas DESC </pre>
Fila 3 – Pregunta de Negocio 3: Conversión de compradores	<pre> SELECT StreamersRegistrados FROM v_ConversionCompradoresAssets SELECT StreamersCompradoresUnicos FROM v_ConversionCompradoresAssets SELECT PorcentajeConversionTienda FROM v_ConversionCompradoresAssets </pre>
Fila 4 – Pregunta de Negocio 4: Picos de tráfico	<pre> SELECT DiaSemanaNombre, PromedioAlertasPorDia, TotalAlertasHistoricas FROM v_PicosTráficoAlertas ORDER BY DiaSemanaNumero </pre>
Fila 5 – Pregunta de Negocio 5: Upgrades y downgrades	<pre> SELECT CantidadUpgrades, CantidadDowngrades, CambiosMismoPrecio, TotalCambiosRegistrados FROM v_UpgradesDowngradesSuscripciones </pre>
Fila 6 – Pregunta de Negocio 6: Facturación mensual	<pre> SELECT NombreMes, NumeroMes, TotalFacturado, CantidadVentas, RankingVentasMonto FROM v_TendenciaVentasMensual ORDER BY NumeroMes </pre>
Fila 7 – Pregunta de Negocio 7: Ventas por semana del mes	<pre> SELECT SemanaMesNombre, NumeroSemanaMes, CantidadVentas, TotalFacturado, RankingVentas FROM v_VentasPorSemanaMes </pre>

	ORDER BY NumeroSemanaMes
Fila 8 – Pregunta de Negocio 8: Suscripciones por semana del mes	<pre> SELECT SemanaMesNombre, NumeroSemanaMes, CantidadSuscripciones FROM v_SuscripcionesPorSemanaMes ORDER BY NumeroSemanaMes </pre>
Fila 9 – Stored Procedures	<pre> EXEC sp_BuscarComprasAssets @FechaInicio = '2025-01-01', @FechaFin = '2026-12-31', @TipoAsset = NULL --- EXEC sp_ReporteDonacionesRango @MontoMinimo = 1.00, @MontoMaximo = 999999.99 --- EXEC sp_FichaRendimientoStreamer @IdUsuario = \${idUsuario} </pre>

3. Diseño de la capa de acceso - criterios

3.1 ¿Cuándo conviene una vista y cuándo un SP?

Característica	Vista	SP
El resultado depende solo del estado de la base, sin parámetros del usuario	Sí	No
Necesita parámetros (rangos, filtros, IDs específicos)	No	Sí
Devuelve un único result set	Sí	Sí
Requiere validación de entrada con mensaje de error	No	Sí
Se compone con otras queries mediante JOIN o subqueries	Sí	No
El dashboard necesita aplicar su propio ORDER BY o WHERE sobre el resultado	Sí	No

En este proyecto, las ocho preguntas de negocio se responden con **vistas** porque sus resultados no dependen de parámetros del usuario: **siempre muestran el estado completo de la base**. Los tres SPs complementan esas vistas ofreciendo consultas parametrizadas para exploración puntual: filtrar compras por período, consultar la ficha de un streamer específico, o analizar donaciones dentro de un rango de montos.

3.2 ¿Cuándo conviene un CTE y cuándo una subconsulta?

La consigna exige mínimo un uso de CTE. En este proyecto se usan cuatro, cada uno con un propósito genuino diferente:

`v_RequestAlertasPorSuscripcion` usa un CTE para resolver el problema de fan-out: un usuario con historial de upgrades y downgrades tiene múltiples filas en `Suscripciones`. Un JOIN directo con esa tabla contaría cada evento de alerta N veces. El CTE encapsula el `ROW_NUMBER()`

que selecciona la suscripción más reciente de cada usuario antes de hacer el JOIN, resolviendo el problema en una sola lectura sin subconsultas correlacionadas anidadas.

`v_ConversionCompradoresAssets` usa dos CTEs independientes en secuencia para calcular el total de usuarios registrados y el total de compradores únicos por separado, y luego combinarlos en el SELECT final. Sin CTEs esto requeriría dos subconsultas escalares en el SELECT, más difíciles de leer y de mantener.

`v_PicosTraficoAlertas` requiere dos niveles de agrupación: primero por fecha real del calendario, después por día de semana. Un GROUP BY no puede hacer ambos niveles en la misma pasada. El CTE separa el nivel 1 (alertas por jornada concreta) del nivel 2 (promedio por día de semana), haciendo la lógica explícita y legible.

`v_UpgradesDowngradesSuscripciones` usa un CTE para encapsular el cálculo de LAG(). Sin él, las expresiones de ventana tendrían que repetirse dentro de cada condición CASE WHEN, lo cual además de repetitivo haría que SQL Server evaluara la función de ventana múltiples veces.

4. Mapa de los 8 indicadores

Pregunta de Negocio	Indicador	Visualización	Origen
¿Qué tipos de recursos prefieren comprar los streamers?	Ventas e ingresos por tipo y por producto	Barras verticales por tipo + tabla de ranking de productos	<code>v_PopularidadAssetsTienda</code>
¿Qué porcentaje de alertas corresponde a cada plan?	Distribución porcentual de eventos por plan	Gráfico de torta + tabla con porcentajes	<code>v_RequestAlertasPorSuscripcion</code>
¿Qué porcentaje de usuarios compra en la tienda?	Tasa de conversión de registrados a compradores	Stat panel con porcentaje + comparación registrados vs compradores	<code>v_ConversionCompradoresAssets</code>
¿Qué días de la semana concentran más actividad?	Promedio de alertas por día de semana y total de alertas históricas	Barras horizontales ordenadas por día	<code>v_PicosTraficoAlertas</code>

¿Cuántos usuarios cambian de plan y en qué dirección?	Conteo de upgrades vs downgrades	Stat panels comparativos	v_UpgradesDowngradesSuscripciones
¿En qué meses se factura más y menos?	Facturación mensual con ranking	Barras verticales por mes + tabla de ranking de productos	v_TendenciaVentasMensual
¿En qué semana del mes se factura más?	Facturación por semana del mes	Barras horizontales por semana + tabla de ranking de productos	v_VentasPorSemanaMes
¿En qué semana del mes se activan más suscripciones?	Altas de suscripciones por semana del mes	Barras verticales por semana	v_SuscripcionesPorSemanaMes

5. Justificación técnica de la capa de acceso

5.1 Función escalar *fn_SemanaMes* (opcional — incluida por buena práctica)

La lógica de clasificación de un día del mes en una de las cuatro semanas (CASE WHEN DATEPART(DAY) <= 7 THEN 1 . . .) aparece de forma idéntica en v_VentasPorSemanaMes y v_SuscripcionesPorSemanaMes. Extraerla a una función escalar aplica el principio de no repetición: si el criterio de segmentación cambia, se modifica en un único lugar y ambas vistas lo heredan automáticamente. Sin la función, un cambio de criterio requeriría modificar dos vistas por separado con riesgo de inconsistencia entre ellas.

5.2. Funciones de ventana — por qué cada una y no una alternativa

SUM(COUNT(*)) OVER() en v_RequestAlertasPorSuscripcion: calcula el total global de alertas en la misma pasada que el GROUP BY. La alternativa sería una subconsulta escalar (SELECT COUNT(*) FROM AlertasEventos), que requiere una lectura adicional de la tabla completa por cada fila del resultado. La función de ventana elimina esa lectura extra.

LAG() en v_UpgradesDowngradesSuscripciones: accede al valor de la fila anterior dentro del mismo usuario, ordenada por fecha de inicio. La alternativa sería un self-JOIN de Suscripciones consigo misma para emparejar cada suscripción con la anterior, lo que produce un plan de ejecución más costoso y un código considerablemente más difícil de leer y mantener.

RANK() en vistas y SPs de facturación: se usa RANK() en lugar de ROW_NUMBER() para que filas con el mismo valor total compartan posición (empate real). Con ROW_NUMBER(), dos meses

con idéntica facturación recibirían posiciones distintas de forma arbitraria, lo cual es semánticamente incorrecto para un ranking de negocio.

`ROW_NUMBER()` en el CTE de `v_RequestAlertasPorSuscripcion`: se usa específicamente `ROW_NUMBER()` en lugar de `RANK()` porque el objetivo es seleccionar exactamente una suscripción por usuario (la más reciente). Con `RANK()`, si dos suscripciones tuvieran la misma `FechaInicio` ambas recibirían `rn = 1` y el `JOIN AND u.rn = 1` devolvería dos filas para ese usuario, reintroduciendo el fan-out que el CTE buscaba eliminar.

5.3 Índices — justificación del diseño cubriente con `INCLUDE`

Los tres índices usan la cláusula `INCLUDE` para agregar columnas que no forman parte de la clave de búsqueda pero sí son proyectadas por las consultas que usan el índice. Esto convierte a cada índice en un índice cubriente: el motor puede resolver la consulta completa leyendo solo las páginas del índice, sin necesidad de volver a la tabla base mediante un Key Lookup por cada fila encontrada.

`IX_Suscripciones_IdUsuario_FechaInicio` tiene un impacto especial: el orden de columnas (`IdUsuario`, `FechaInicio`) es deliberado. Las funciones de ventana `LAG()` `OVER (PARTITION BY IdUsuario ORDER BY FechaInicio)` y `ROW_NUMBER()` `OVER (PARTITION BY IdUsuario ORDER BY FechaInicio DESC)` requieren que los datos estén ordenados exactamente así. Sin el índice, SQL Server agrega un operador `SORT` explícito al plan de ejecución, que materializa toda la tabla en memoria para ordenarla, la operación más costosa en términos de CPU y memoria dentro del plan. Con el índice, los datos ya vienen en el orden requerido y el operador `SORT` desaparece del plan.

5.4 SPs — qué problema resuelve cada uno y qué aportan al proyecto

`sp_BuscarComprasAssets` resuelve un problema que ninguna vista puede cubrir: acotar el análisis de ventas a una ventana temporal definida por quien consulta el dashboard.

`v_TendenciaVentasMensual` y `v_VentasPorSemanaMes` muestran patrones ya consolidados sobre la totalidad del historial, pero no permiten aislar, por ejemplo, "qué se vendió en las últimas dos semanas" o "qué asset lideró las ventas de audio en marzo". Al aceptar `@FechaInicio`, `@FechaFin` y opcionalmente `@TipoAsset`, el SP convierte una pregunta exploratoria puntual en una consulta parametrizada contra la capa de acceso, en lugar de forzar un `SELECT` directo a la tabla. Esto amplía el valor real del dashboard: deja de mostrar únicamente el estado agregado de la plataforma y habilita investigación dirigida sobre subconjuntos concretos de datos.

`sp_FichaRendimientoStreamer` resuelve un problema distinto: consolidar en una sola consulta toda la actividad de un streamer puntual (plan, proyectos, alertas, donaciones recibidas, compras realizadas). Este tipo de "ficha 360°" no tiene sentido como vista, porque calcularla para los 60 usuarios simultáneamente implicaría evaluar seis subconsultas correlacionadas por cada fila del resultado, un costo que crece con la base de usuarios sin que el dashboard necesite ese nivel de detalle para todos a la vez. El SP resuelve exactamente el patrón de uso real: un analista hace clic sobre un streamer específico en cualquier panel y necesita su perfil completo bajo demanda.

Es la única pieza de la capa de acceso pensada para consulta unitaria en lugar de consulta poblacional, y por eso complementa lo que ya cubren las ocho vistas.

sp_ReporteDonacionesRango resuelve el problema de segmentar el comportamiento de donación por magnitud de monto, algo que ninguna vista responde:
v_RequestAlertasPorSuscripcion mide carga operativa por plan, no perfil de donantes. Al parametrizar @MontoMinimo y @MontoMaximo, el SP permite comparar, por ejemplo, cuántas donaciones pequeñas (la franja que en el Entregable 2 se definió como el 75% de los eventos tip) conviven con las donaciones grandes, minoritarias pero de mayor impacto económico. Esto le da al dashboard una herramienta para preguntas derivadas que no estaban entre las ocho originales pero surgen naturalmente del análisis: ¿el ingreso por tips se sostiene en volumen de donaciones chicas o en pocos donantes grandes?

En conjunto, los tres SPs no compiten con las vistas: atienden una necesidad que las vistas, por diseño, no pueden cubrir, la exploración con parámetros elegidos en el momento de la consulta. Cumplen así no solo el mínimo cuantitativo de la consigna (3 SPs con parámetros funcionales), sino su intención declarada: sostener tanto el reporte fijo (vistas, preguntas de negocio predefinidas en el Entregable 1) como la exploración dirigida (SPs, preguntas que quien usa el dashboard formula en el momento).

6. Conclusión

El desarrollo de MagicDB permitió recorrer el ciclo completo que plantea la consigna: partir de un caso de uso real, una plataforma SaaS para streamers, y llevarlo, sin atajos, desde el modelo conceptual hasta un dashboard funcional que responde preguntas de negocio concretas. Cada decisión tomada a lo largo de los tres entregables tuvo una razón trazable: el recorte deliberado del alcance para evitar datos sensibles, la normalización de EstadosSuscripcion y TiposAlerta, la elección de NO ACTION sobre datos con valor histórico y CASCADE sobre datos dependientes del usuario, y la división entre vistas para preguntas fijas y stored procedures para exploración parametrizada.

El proyecto no solo alcanzó los mínimos cuantitativos de la consigna, sino que los superó en varios frentes: 12 tablas en lugar de 10, 8 preguntas de negocio en lugar de 5, 14 vistas en lugar de 5, y 8 indicadores en el dashboard en lugar de 5. Más importante que el número, cada exceso respondió a una necesidad real del caso, por ejemplo, las vistas de facturación por semana del mes (P7 y P8) surgieron porque el análisis mensual (P6) resultaba insuficiente para detectar patrones de estacionalidad más finos, no porque hiciera falta "sumar vistas" para cumplir un mínimo.

La restricción técnica más exigente, que el dashboard no pudiera consultar tablas directamente, terminó siendo la que más ordenó el diseño: obligó a pensar la capa de acceso como un contrato estable entre el motor de base de datos y la herramienta de visualización. Esa disciplina es, en definitiva, la que se busca imitar de un entorno profesional real, donde nadie con acceso al dashboard debería poder alterar ni exponer accidentalmente la estructura interna de la base.

Como limitación reconocida, los datos cargados son sintéticos y las distribuciones (30/30/25/15 en planes, 40% de conversión en la tienda, 70% de eventos desplazados a horario nocturno, entre otras) fueron elegidas por el grupo para que el dashboard mostrara variabilidad interpretable, no observadas de una plataforma real. Esto es coherente con el objetivo académico del trabajo, pero es importante remarcarlo en la defensa oral: los indicadores muestran cómo se comportaría el modelo de datos ante un patrón de uso razonable, no un diagnóstico real de negocio. Si el proyecto se llevara a producción, el paso natural sería reemplazar la carga procedural por datos reales de la plataforma y revisar si las distribuciones supuestas siguen siendo representativas.

En síntesis, MagicDB cumple con lo que pide la consigna en su punto más importante: cada decisión de modelado, indexación y diseño de la capa de acceso puede explicarse y defenderse individualmente, con una razón de negocio o técnica detrás, y no como una elección arbitraria.